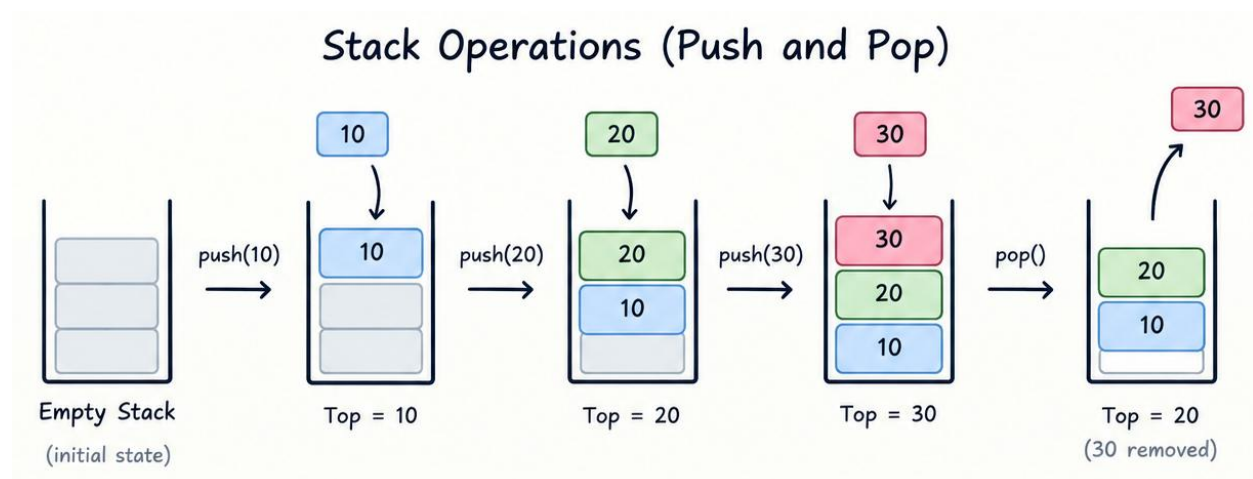


Stack

Stack is a linear data structure that stores elements in a specific order. It follows the Last In, First Out (LIFO) principle, which means that the most recently added element is the first one to be removed. A simple real-world example is a stack of plates. When a new plate is added, it is placed on top, and when a plate is removed, the top plate must be taken first. A stack provides access to its top element, with insertion and removal normally performed only at the top. Although the underlying implementation may allow direct access to other elements, doing so bypasses the standard stack operations and LIFO behavior. This makes stacks useful in situations where the most recently added item needs to be processed first, such as undo operations, function calls, expression evaluation, and browser history.

For example, if we push the values 10, 20, and 30 onto a stack, the stack will contain 10 at the bottom, followed by 20, with 30 at the top. If we perform a pop operation, 30 will be removed first. The next pop will remove 20, followed by 10. Therefore, the removal order is the reverse of the insertion order: $30 \rightarrow 20 \rightarrow 10$.



Stack Operations

A stack provides several basic operations that allow elements to be added, removed, accessed, and checked. The exact operations available can depend on the implementation, but the most common ones are `push()`, `pop()`, `peek()` (or `top()`), `isEmpty()`, `isFull()`, and `displayAll()`. These operations work together to maintain the LIFO behavior of the stack.

1. push()

The `push()` operation is used to add a new element to the top of the stack. Whenever an element is pushed, it becomes the new top element.

For example, if the stack contains:

Top → 30 → 20 → 10

Performing push(40) results in:

Top → 40 → 30 → 20 → 10

Time Complexity : $O(1)$

2. pop()

The pop() operation removes the element currently at the top of the stack. Since a stack follows LIFO, the most recently added element is always removed first.

Using the above example, if a pop() is performed, the top element 40 is removed:

After pop():

Top → 30 → 20 → 10

The removed element can also be returned by the function, if needed.

Time Complexity: $O(1)$

3. peek() / top()

The top() operation returns the element at the top of the stack without removing it. This is useful when we want to know what element would be removed next while keeping the stack unchanged.

If a stack has:

Top → 30 → 20 → 10

Performing top() or peek() returns 30.

Time Complexity: $O(1)$

4. isEmpty()

This operation checks whether the stack currently contains any elements. It returns true if the stack has no elements and false otherwise.

For example:

Stack:

(empty)

isEmpty() → true

Perform push(10):

Top → 10

isEmpty() → false

Time Complexity: $O(1)$

5. isFull()

The isFull() operation checks whether the stack has reached its maximum capacity. This operation is mainly used when the stack is implemented using a fixed-size array.

For example, if the maximum capacity is 5:

Top → 50

40

30

20

10

The stack is full, so:

isFull() → true

If the stack has fewer than 5 elements, isFull() returns false.

For dynamically implemented stacks, such as one using a linked list, a traditional isFull() check is generally not needed because the stack can grow as memory is available.

Time Complexity: $O(1)$

6. displayAll()

The displayAll() operation is generally a custom/helper operation rather than a fundamental Stack ADT operation. It is used to display all the elements currently stored in the stack.

For example:

Stack:

Top → 40

30

20

10

displayAll() might produce:

40 30 20 10

The elements are normally displayed starting from the top so that the current stack order is easy to understand. Unlike pop(), displaying the elements should not remove them from the stack.

Time Complexity: $O(n)$, because every element needs to be visited.

Operation	Purpose	Changes Stack?	Time Complexity
push()	Adds an element to the top	Yes	O(1)
pop()	Removes the top element	Yes	O(1)
peek()/top()	Views the top element	No	O(1)
<u>isEmpty()</u>	Checks whether stack is empty	No	O(1)
<u>isFull()</u>	Checks whether fixed-size stack is full	No	O(1)
<u>displayAll()</u>	Displays all elements	No	O(n)

Stack Overflow

Stack overflow occurs when we try to push a new element into a stack that is already full. For example, suppose a stack has a maximum capacity of 3:

Top → 30

20

10

If we try:

push(40);

there is no available space for the new element. Therefore, the operation results in stack overflow.

Stack Capacity = 3

Current Size = 3

push(40) → STACK OVERFLOW

Overflow is particularly relevant when the stack is implemented using a fixed-size array. A dynamically allocated stack, such as a linked-list implementation, does not have a fixed capacity in the same way; it can generally continue growing until available memory is exhausted.

Stack Underflow

Stack underflow occurs when we try to remove or access an element from an empty stack.

For example:

Stack:

(empty)

If we perform:

`pop();`

there is no element available to remove, so the operation results in stack underflow.

The same issue can occur if `peek()` or `top()` is called on an empty stack, depending on how the implementation handles the situation.

Stack Size = 0

`pop()` → STACK UNDERFLOW

Array-based implementation using a static array

A stack can be implemented using a static array, where the size of the stack is fixed when the program is created. In this approach, the array stores the elements of the stack, while a variable called `top` keeps track of the position of the top element.

For example, if we create a stack with a capacity of 5, the array has five available positions. Initially, `top` can be set to -1 to indicate that the stack is empty.

```
int stack[5];
```

```
int top = -1;
```

When an element is pushed, `top` is increased and the element is placed at that position. When an element is popped, the element at `top` is removed and `top` is decreased. Since insertion and removal happen only at the top, both operations take $O(1)$ time.

A static array implementation also makes it easy to detect overflow. If `top` reaches capacity - 1, the stack is full and another `push()` cannot be performed. Similarly, if `top == -1`, the stack is empty, and attempting to `pop()` results in underflow.

```

Stack.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Stack{
5  private:
6      int stack[5];
7      int top;
8
9  public:
10     Stack() {
11         top = -1;
12     }
13
14     void push(int value) {
15         if(top == 4) {
16             cout << "Stack Overflow";
17             return;
18         }
19         stack[++top] = value;
20     }
21
22     void pop() {
23         if (top == -1) {
24             cout << "Stack Underflow" << endl;
25             return;
26         }
27         top--;
28     }
29
30     int peek() {
31         if (top == -1) {
32             cout << "Stack is Empty" << endl;
33             return -1;
34         }
35     }
36
37
38     return stack[top];
39 }
40
41 bool isEmpty() {
42     return top == -1;
43 }
44
45 bool isFull() {
46     return top == 4;
47 }
48
49 void displayAll() {
50     if (top == -1) {
51         cout << "Stack is Empty" << endl;
52         return;
53     }
54     cout << "Stack elements: ";
55
56     for (int i = top; i >= 0; i--) {
57         cout << stack[i] << " ";
58     }
59     cout << endl;
60 }
61
62 };
63
64
65
66

```

Advantages and Limitations of an array-based stack

Advantages

- Simple to implement and understand.
- Fast access to the top element.
- Push(), pop(), and peek() take $O(1)$ time.
- Does not require dynamic memory allocation.

Limitations

The main limitation is the fixed size. If the array has a capacity of 5, it cannot store a sixth element without changing the implementation. This can lead to stack overflow even if the computer has plenty of available memory.